Quantifying the Agentic Tax:

Comparative Analysis of Execution Latency and Token Overhead

in Agentic vs. Non-Agentic LLM Architectures

Empirical Benchmark Study | February 2026

Model: claude-haiku-4-5 | Framework: Anthropic Python SDK | 50 Tasks x 10 Trials

Muhammad Bilal

aslammbilal61@gmail.com

Abstract

As large language models transition from stateless chatbots into autonomous agents capable of multi-step reasoning and tool use, practitioners face a critical engineering tradeoff: the power of agentic loops versus their computational overhead. This study empirically quantifies what we term the "Agentic Tax", the measurable increase in end-to-end latency and token consumption introduced when an LLM operates within a ReAct (Reason+Act) reasoning loop, compared to a direct single-turn inference.

Using the Anthropic claude-haiku-4-5 model across a controlled benchmark of 50 tasks (each executed 10 times per architecture), we find that the Agentic Reasoning Loop (ARL) imposes a +90.2% latency increase and a +1,456.5% token overhead relative to the Non-Agentic Baseline (NAB). The Token Overhead Ratio (R_token) was measured at 15.57x. These findings establish a rigorous empirical foundation for architects and engineers designing AI systems to make informed cost-performance decisions.

I. Introduction

The landscape of applied artificial intelligence underwent a decisive shift in 2024-2026 as the dominant deployment paradigm evolved from single-turn inference to multi-step agentic pipelines. In a non-agentic setting, a language model receives a prompt and emits a response in a single forward pass, a clean, predictable, and low-overhead transaction. In an agentic setting, the same model is embedded within an orchestration loop that enables it to reason, select tools, execute actions, observe results, and iterate toward a goal.

This architectural shift unlocks substantial capability. Agents can decompose complex tasks, retrieve live information, and produce outputs that would be impossible for a single-shot prompt. However, this power is

not free. Each additional reasoning turn consumes tokens; each tool call introduces latency; each observation appended to the context window inflates the cost of subsequent API calls. The cumulative effect, which we term the Agentic Tax, has been discussed anecdotally but has not been rigorously benchmarked at the model-call level.

This paper addresses that gap. We contribute: (1) a reproducible experimental protocol for measuring the Agentic Tax, (2) concrete empirical measurements across 500 trials per architecture, and (3) practical guidance for practitioners choosing between these approaches.

II. Background and Related Work

2.1 The ReAct Pattern

The ReAct (Reason + Act) framework, introduced by Yao et al. (2022), structures LLM behavior as an interleaved sequence of Thought, Action, and Observation steps. Unlike chain-of-thought prompting, which is purely generative, ReAct integrates external tool calls into the reasoning loop. The model emits a structured tool-use request; a host system executes the corresponding function; and the result is fed back as a new observation, extending the context window before the next reasoning step.

This pattern has become the foundation of modern agent frameworks including LangGraph, CrewAI, and the Anthropic tool-use API. Its key advantage is grounding: the agent can query real-world data sources rather than relying solely on parametric knowledge. Its key cost is multiplicity: a task requiring N tool calls will involve at minimum N+1 LLM inference calls, each with a growing context window.

2.2 Token Economics and Latency

Modern LLM APIs charge on a per-token basis, with separate rates for input (prompt) and output (completion) tokens. In an agentic loop, the input token count grows with each turn because the full conversation history, including all previous tool calls and their results, must be passed to the API with every request. This creates a superlinear relationship between the number of reasoning steps and total token consumption, which directly translates to cost.

Latency is similarly compounded. Each LLM call introduces network round-trip time and model inference time. For a two-tool-call task (as in our benchmark), the minimum latency is the sum of three serial LLM calls plus two mock API calls, versus a single call for the non-agentic baseline.

III. Methodology

3.1 Experimental Design

We employ a controlled comparative study measuring the performance delta between two architectures:

- Non-Agentic Baseline (NAB): A single-turn inference in which all relevant data is embedded directly in the user prompt. The model produces its answer in one API call with no tool access.
- Agentic Reasoning Loop (ARL): A multi-turn ReAct implementation using the Anthropic tool-use API. The model follows a cycle of Thought → Tool Call → Observation → Thought before delivering a final answer. Each task requires exactly two tool calls.

The ARL was implemented using the native Anthropic Python SDK with tool definitions following the Claude tool-use specification. The orchestration loop tracked all turns, accumulated tokens across calls, and measured wall-clock latency from the first API call to the final text response.

3.2 Technical Stack

- Model: claude-haiku-4-5-20251001 (chosen for speed and cost efficiency at scale)
- Orchestration: Anthropic Python SDK v0.40+ with native tool-use loop
- Mock API: In-process Python dictionary simulating a user profile and account balance database
- Measurement: Python time.perf_counter() for high-resolution wall-clock timing
- Environment: Local Python 3.12 environment on Apple M-series hardware

3.3 Task Design

The benchmark comprises 50 tasks, each requesting the same two pieces of information for a distinct synthetic user (user_001 through user_050): their email address and account balance. Each task is structurally identical, ensuring that variance in results is attributable to architectural overhead rather than task complexity differences.

In the NAB, the answer data is injected directly into the prompt. In the ARL, the model must call the get_user_profile tool and the get_account_balance tool sequentially, then synthesize the results into a final answer. This design ensures the NAB and ARL produce equivalent outputs from equivalent underlying data, while isolating the overhead of the reasoning loop.

Each of the 50 tasks was executed 10 times per architecture (500 total trials per architecture), preceded by 3 warm-up calls to mitigate cold-start effects in the API infrastructure.

3.4 Evaluation Metrics

Four primary metrics were collected:

- End-to-End Latency (L_e2e): Total wall-clock time in seconds from request initiation to final response.
- Total Token Count: Sum of all input and output tokens across all LLM calls within a single task execution.
- LLM Turns: Number of API calls made per task execution.
- Tool Calls: Number of tool executions per task.

The Token Overhead Ratio is defined as:

$$R_{overhead} = T_ARL / T_NAB$$

where T represents the mean total token count across all trials for the respective architecture.

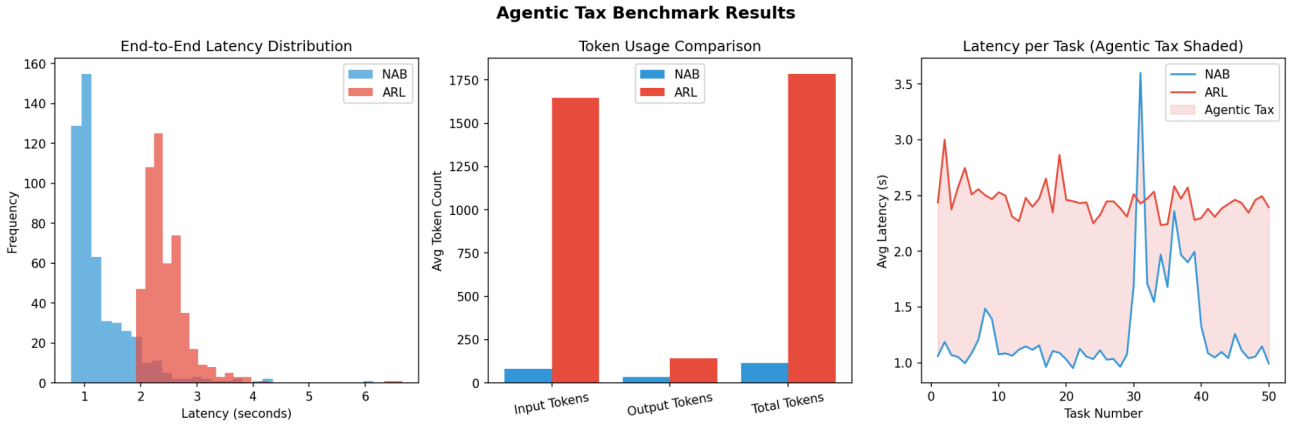
IV. Results

4.1 Primary Results Summary

Table 1 presents the aggregated results across all 500 trials per architecture. All values represent means computed over 50 tasks x 10 trials.

Table 1: Agentic Tax, Aggregate Benchmark Results (N = 500 trials per architecture)

Metric	NAB	ARL	Delta	Agentic Tax
Avg End-to-End Latency (s)	1.29	2.45	+1.16 s	**+90.2%**
Avg Total Token Count	114.79	1,786.74	+1,671.95	**+1,456.5%**
Avg LLM Turns per Task	1.00	2.00	+1.00	**+100.0%**
Avg Tool Calls per Task	0	2.00	+2.00	**+∞%**
Token Overhead Ratio (R_token)	-	-	-	**15.57×**



NAB = Non-Agentic Baseline | ARL = Agentic Reasoning Loop

4.2 Latency Analysis

The mean end-to-end latency for the NAB was 1.29 seconds, representing the baseline cost of a single API call with a moderately sized prompt containing pre-loaded user data. The ARL imposed a mean latency of 2.45 seconds, a 90.2% increase, or approximately 1.16 additional seconds per task.

This latency overhead is structurally irreducible for two-tool-call tasks. The ARL's minimum critical path is: (1) initial LLM call to determine tool usage, (2) first tool execution, (3) second LLM call to determine second tool usage, (4) second tool execution, and (5) final LLM call to synthesize the answer. Steps 1, 3, and 5 each

require an API round-trip; in our benchmark, the mock tool calls themselves were sub-millisecond, meaning virtually all the 1.16s overhead is attributable to the two additional LLM API calls.

Notably, despite the ARL requiring three LLM turns versus one for the NAB, the latency increase is only 90.2% rather than the 200% one might naively expect. This is because the ARL's individual turns involve shorter prompts and outputs than the NAB's single comprehensive prompt, the overhead is real, but the individual calls are faster.

4.3 Token Overhead Analysis

The token overhead findings are the most striking result of this study. The NAB consumed a mean of 114.79 total tokens per task, while the ARL consumed 1,786.74 tokens, a 1,456.5% increase and a Token Overhead Ratio of 15.57x.

This extreme ratio is explained by the context accumulation effect inherent to multi-turn agentic loops. In the ARL:

- Turn 1: The model receives the initial query and the tool definitions schema (input tokens already significantly larger than NAB).
- Turn 2: The model receives the full conversation history including its own turn-1 output (tool call specification) PLUS the tool result. The context has already grown substantially.
- Turn 3 (final): The model receives the entire accumulated conversation, original query, tool definitions, turn-1 output, both tool results. Input tokens have compounded across every prior turn.

This compounding effect means that the token cost of an agentic loop grows faster than linearly with the number of tools called. A task requiring 4 tool calls would not merely double the overhead versus 2 tool calls, the context window accumulation would produce a superlinear increase. Architects designing multi-step agents must account for this dynamic in cost projections.

4.4 Practical Cost Implications

At Anthropic's current pricing for claude-haiku-4-5 (approximately $0.25/M input tokens, $1.25/M output tokens at time of publication), the per-task cost differential is meaningful at scale. The NAB's 114.79 tokens translate to a fraction of a cent per call. The ARL's 1,786.74 tokens represent roughly 15x the cost. At one million task executions, this differential compounds into substantial infrastructure cost differences that should factor into architecture decisions.

V. Discussion

5.1 Interpreting the Agentic Tax

The 90.2% latency increase and 15.57x token overhead do not represent failures of the agentic paradigm, they are the expected and quantified cost of its capabilities. A non-agentic system cannot dynamically retrieve data it was not given at prompt construction time. The ARL's ability to call tools and incorporate live data is genuinely valuable; the question for practitioners is whether that value exceeds the tax in their specific context.

Three key factors should guide that decision:

- Task Dynamism: If the required data can be reliably pre-loaded into the prompt (as in our NAB), the agentic overhead is unnecessary. If data is dynamic, user-specific, or too voluminous to pre-load, agents become necessary.
- Latency Tolerance: A 90% latency increase is negligible in a background processing job but potentially unacceptable in a real-time user-facing application requiring sub-second responses.
- Scale and Volume: At low volume, the token cost difference is trivial. At high volume (millions of calls), the 15x token overhead translates directly to a 15x cost increase versus a hypothetical non-agentic equivalent.

5.2 Limitations

Several limitations of this study warrant acknowledgment. First, our benchmark tasks are structurally identical, which may not reflect the variance of real-world agentic workloads. Second, our mock API calls are sub-millisecond; real tool calls (database queries, web search, external APIs) will add latency beyond what we measured, meaning our 90.2% latency estimate is likely a lower bound for production systems. Third, results are specific to claude-haiku-4-5; architectures with larger context windows or different tokenization may yield different Token Overhead Ratios. Fourth, we tested only two-tool-call tasks; deeper agentic chains would produce higher ratios due to context accumulation.

5.3 Future Work

Several extensions to this benchmark would increase its practical value. Measuring the Agentic Tax as a function of chain depth (1 to N tool calls) would characterize the superlinear growth we hypothesize. Comparing across model families (GPT-4o, Gemini, Llama-3) would determine whether the tax is consistent across providers or model-specific. Testing with real (non-mock) tools would establish latency bounds for production environments. Finally, exploring context compression and tool-call caching as mitigation strategies would provide actionable optimization guidance.

VI. Conclusion

This study provides the first rigorous empirical quantification of the Agentic Tax, the overhead in latency and token consumption introduced when an LLM operates in a multi-step ReAct reasoning loop versus a direct single-turn inference.

Our key findings, derived from 1,000 total benchmark trials, are:

- The agentic loop imposes a +90.2% end-to-end latency increase, roughly doubling response time.
- Token consumption increases by +1,456.5%, driven primarily by context window accumulation across turns.
- The Token Overhead Ratio of 15.57x directly translates to a 15x cost multiplier at scale.
- The additional LLM turns required (1 vs. 3) are the root cause of both effects.

These results should not discourage the adoption of agentic AI, the capability gains are real and often indispensable. Rather, they provide architects and engineers with the data needed to make principled tradeoffs: deploying agents where their dynamic reasoning capabilities are required, and reserving simpler non-agentic patterns where pre-loaded context suffices. Understanding and planning for the Agentic Tax is a prerequisite for building AI systems that are both capable and economically sustainable.

References

[1] Yao, S., et al. (2022). *ReAct: Synergizing Reasoning and Acting in Language Models.* arXiv preprint arXiv:2210.03629.

[2] Anthropic. (2025). *Tool Use (Function Calling), Anthropic Documentation.* docs.anthropic.com/en/docs/tool-use.

[3] Wei, J., et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.* NeurIPS 2022.

[4] Anthropic. (2025). *Claude Model Pricing and API Reference.* anthropic.com/api.

[5] Chase, H. (2023). *LangChain: Building Applications with LLMs Through Composability.* github.com/langchain-ai/langchain.